

A Series of Continuous Control Algorithms

Presenter: Hongming Zhang

Date: 2024.02.28

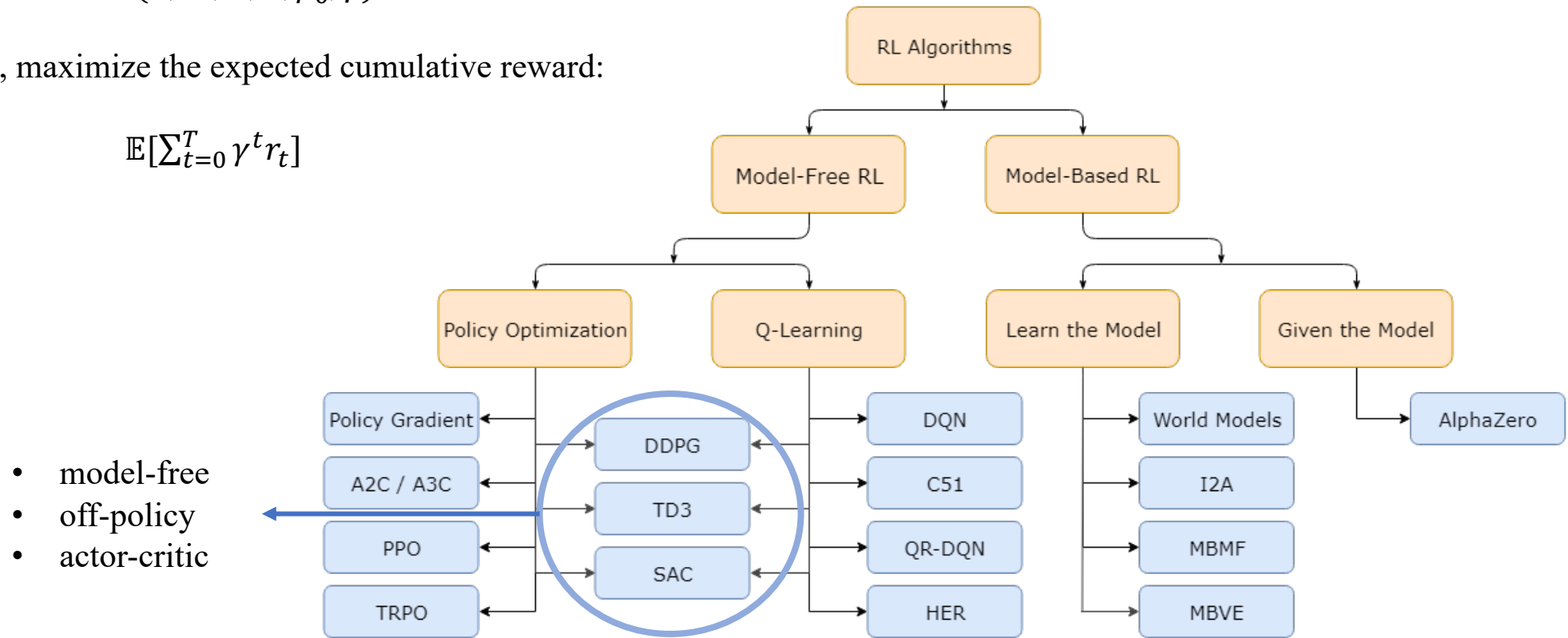
Reinforcement Learning Algorithms

- Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, R, P, \rho_0, \gamma)$$

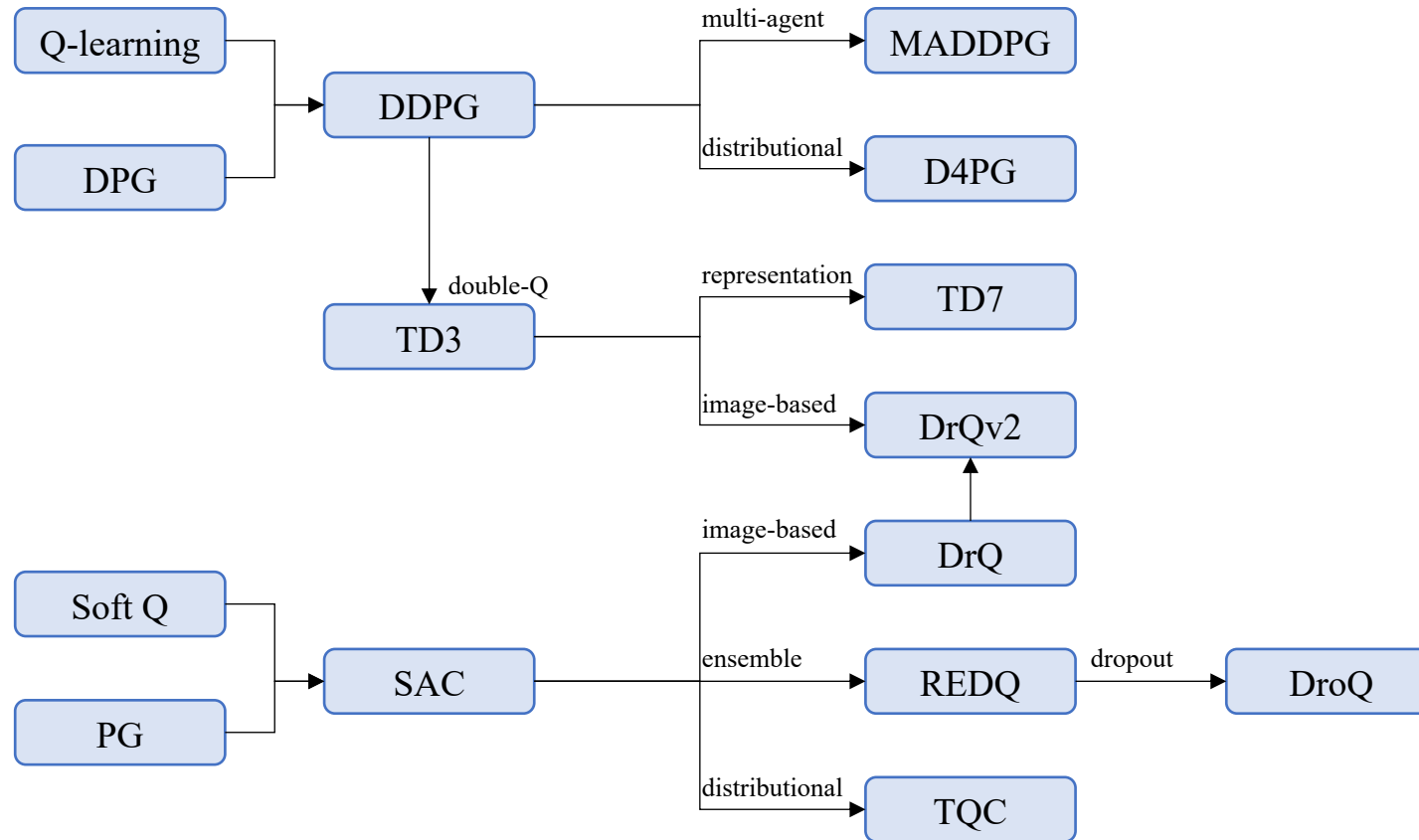
- Objective, maximize the expected cumulative reward:

$$\mathbb{E}[\sum_{t=0}^T \gamma^t r_t]$$



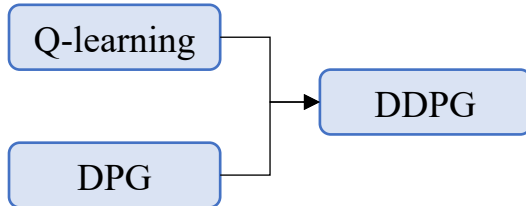
A non-exhaustive, but useful taxonomy of algorithms in modern RL.

Reinforcement Learning Algorithms



Deep Deterministic Policy Gradient (DDPG)

Learn policies in high-dimensional continuous action spaces.



exploration
adding noise sampled from a noise process $\mathcal{N}$

soft target updates

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

DQN

for episode = 1, M **do**

Initialize a random process $\mathcal{N}$ for action exploration

Receive initial observation state s_1

for t = 1, T **do**

Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

Execute action a_t and observe reward r_t and observe new state s_{t+1}

Store transition (s_t, a_t, r_t, s_{t+1}) in R

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

DQN

Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

DPG

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for
end for

Twin Delayed Deep Deterministic Policy Gradient (TD3)

Algorithm 1 TD3

Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ

Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$

Initialize replay buffer $\mathcal{B}$

for $t = 1$ **to** T **do**

 Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$,

$\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'

 Store transition tuple (s, a, r, s') in $\mathcal{B}$

 Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$

$\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$

$y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$

 Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$

if $t \bmod d$ **then**

 Update ϕ by the deterministic policy gradient:

$\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$

 Update target networks:

$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$

$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$

end if

end for

Limit overestimation in DDPG.

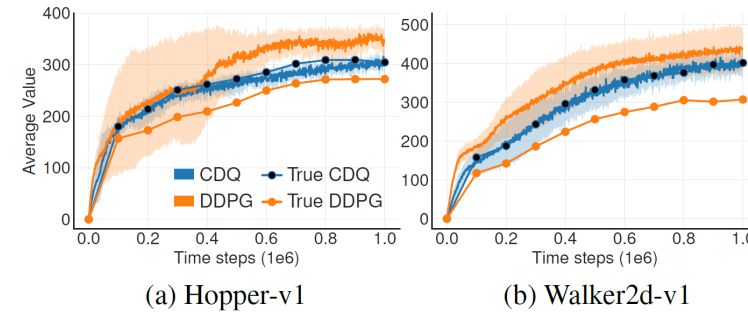


Figure 1. Measuring overestimation bias in the value estimates of DDPG and our proposed method, Clipped Double Q-learning (CDQ), on MuJoCo environments over 1 million time steps.

target policy smoothing

similar actions should have similar value
fitting the value of a small area around the target action

clipped Double Q-learning

an underestimation bias is far preferable to overestimation bias

delaying policy updates

to first minimize value error before introducing a policy update

Twin Delayed Deep Deterministic Policy Gradient (TD3)

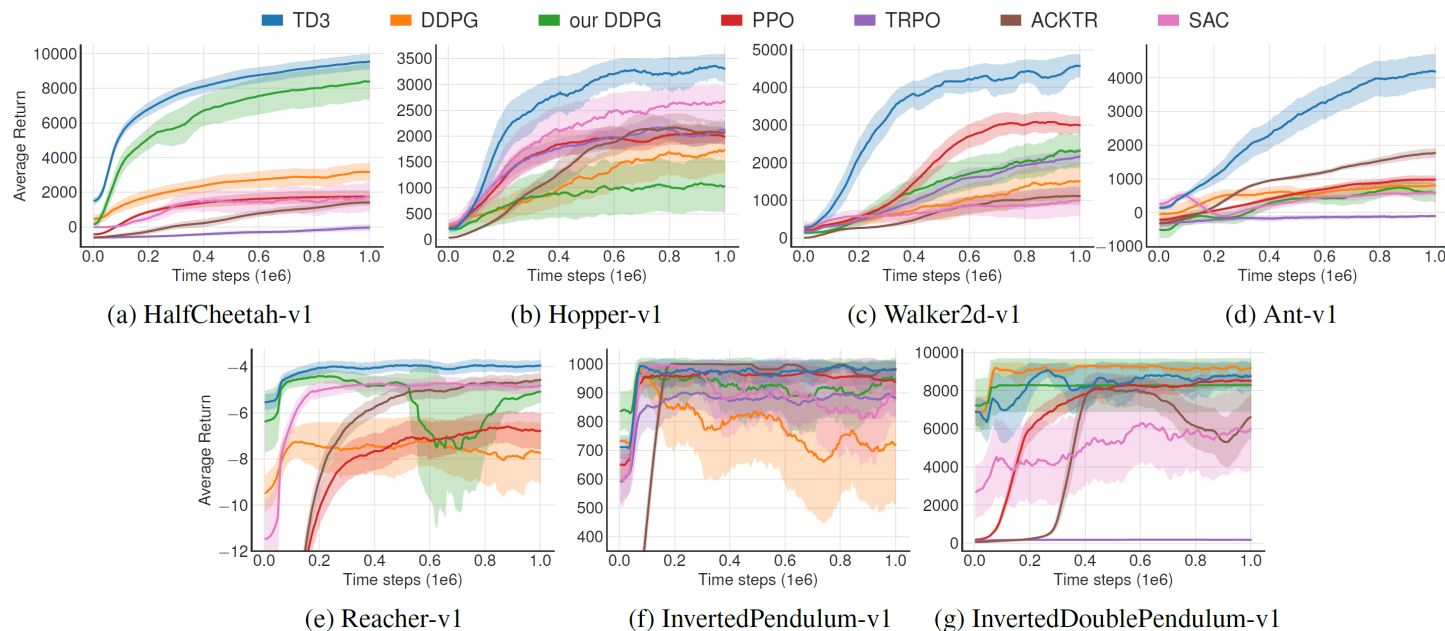


Figure 5. Learning curves for the OpenAI gym continuous control tasks. The shaded region represents half a standard deviation of the average evaluation over 10 trials. Curves are smoothed uniformly for visual clarity.

Table 2. Average return over the last 10 evaluations over 10 trials of 1 million time steps, comparing ablation over delayed policy updates (DP), target policy smoothing (TPS), Clipped Double Q-learning (CDQ) and our architecture, hyper-parameters and exploration (AHE). Maximum value for each task is bolded.

Method	HCheetah	Hopper	Walker2d	Ant
TD3	9532.99	3304.75	4565.24	4185.06
DDPG	3162.50	1731.94	1520.90	816.35
AHE	8401.02	1061.77	2362.13	564.07
AHE + DP	7588.64	1465.11	2459.53	896.13
AHE + TPS	9023.40	907.56	2961.36	872.17
AHE + CDQ	6470.20	1134.14	3979.21	3818.71
TD3 - DP	9590.65	2407.42	4695.50	3754.26
TD3 - TPS	8987.69	2392.59	4033.67	4155.24
TD3 - CDQ	9792.80	1837.32	2579.39	849.75
DQ-AC	9433.87	1773.71	3100.45	2445.97
DDQN-AC	10306.90	2155.75	3116.81	1092.18

Soft Actor-Critic (SAC)



The maximum entropy objective

$$\pi^* = \operatorname{argmax}_{\pi} \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

Modified Bellman backup operator $\mathcal{T}^{\pi}$

$$\mathcal{T}^{\pi} Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})] \quad \text{soft state-action value}$$

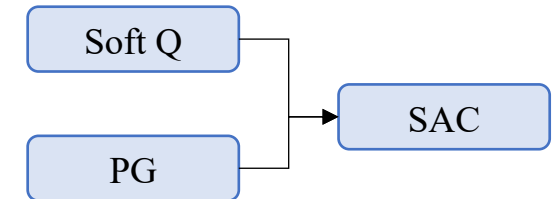
$$V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q(s_t, a_t) - \alpha \log \pi(a_t | s_t)] \quad \text{soft state value}$$

Soft Q-function: minimize the soft Bellman residual

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim D} \left[\frac{1}{2} \left(Q_{\theta}(s_t, a_t) - \left(r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V_{\bar{\theta}}(s_{t+1})] \right) \right)^2 \right]$$

Policy: minimizing the expected KL-divergence

$$J_{\pi}(\phi) = \mathbb{E}_{a_t \sim \pi_{\phi}} [\alpha \log (\pi_{\phi}(a_t | s_t)) - Q_{\theta}(s_t, a_t)]$$



Also incorporate TD3's tricks like
clipped Double Q-learning

Soft Actor-Critic (SAC)

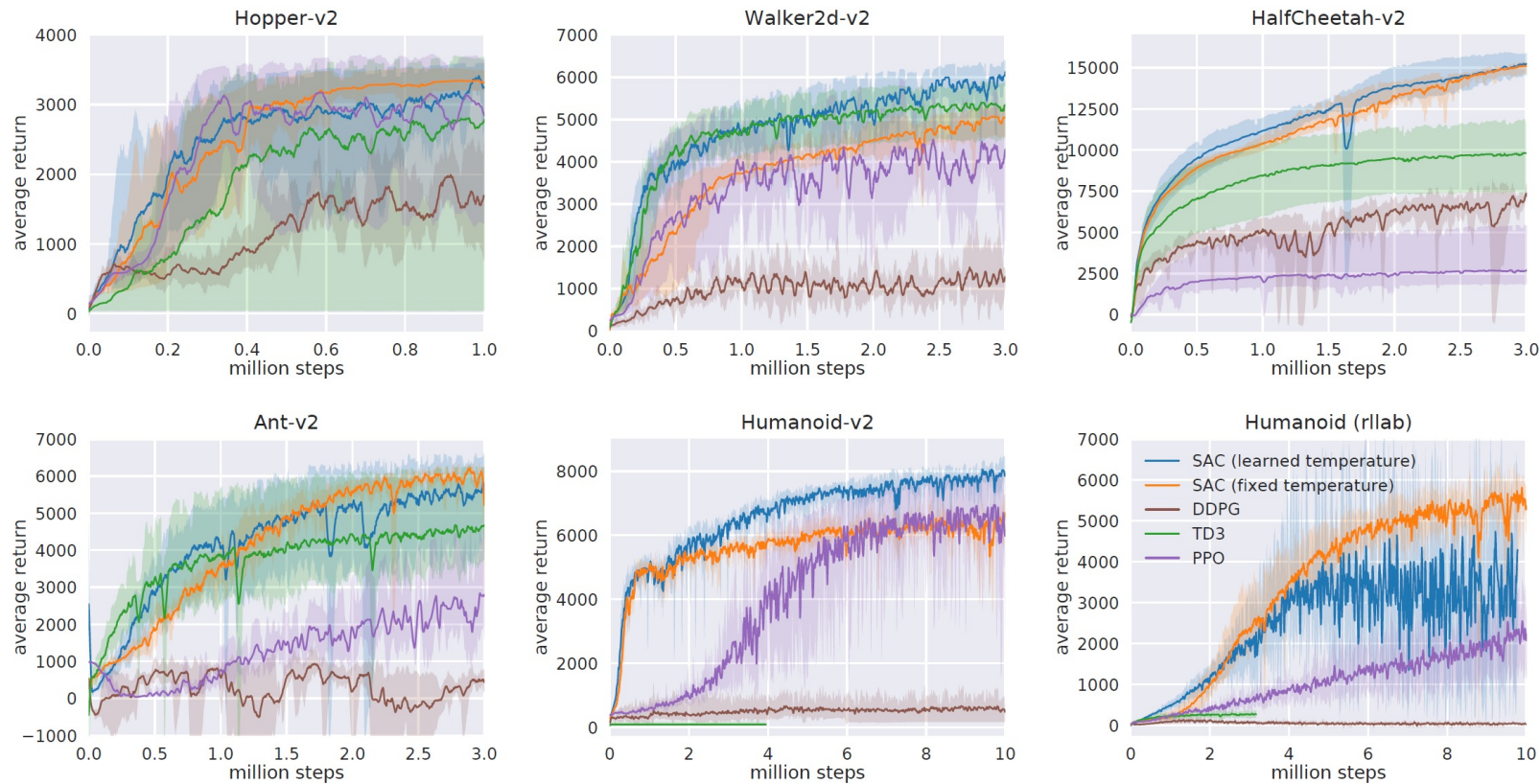


Figure 1: Training curves on continuous control benchmarks. Soft actor-critic (blue and yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

The framework of centralized training with decentralized execution

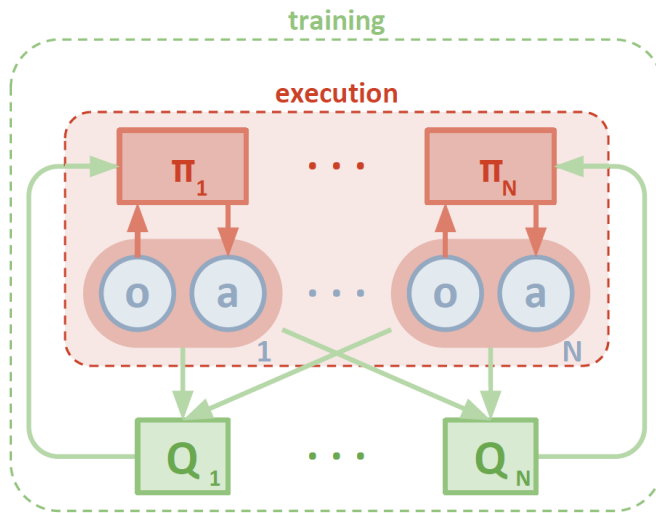


Figure 1: Overview of our multi-agent decentralized actor, centralized critic approach.

Algorithm 1: Multi-Agent Deep Deterministic Policy Gradient for N agents

```

for episode = 1 to  $M$  do
  Initialize a random process  $\mathcal{N}$  for action exploration
  Receive initial state  $\mathbf{x}$ 
  for  $t = 1$  to max-episode-length do
    for each agent  $i$ , select action  $a_i = \mu_{\theta_i}(o_i) + \mathcal{N}_t$  w.r.t. the current policy and exploration
    Execute actions  $a = (a_1, \dots, a_N)$  and observe reward  $r$  and new state  $\mathbf{x}'$ 
    Store  $(\mathbf{x}, a, r, \mathbf{x}')$  in replay buffer  $\mathcal{D}$ 
     $\mathbf{x} \leftarrow \mathbf{x}'$ 
    for agent  $i = 1$  to  $N$  do
      Sample a random minibatch of  $S$  samples  $(\mathbf{x}^j, a^j, r^j, \mathbf{x}'^j)$  from  $\mathcal{D}$ 
      Set  $y^j = r_i^j + \gamma Q_i^{\mu'}(\mathbf{x}'^j, a_1^j, \dots, a_N^j) |_{a_k^j = \mu_k(o_k^j)}$ 
      Update critic by minimizing the loss  $\mathcal{L}(\theta_i) = \frac{1}{S} \sum_j (y^j - Q_i^{\mu}(\mathbf{x}^j, a_1^j, \dots, a_N^j))^2$ 
      Update actor using the sampled policy gradient:
        
$$\nabla_{\theta_i} J \approx \frac{1}{S} \sum_j \nabla_{\theta_i} \mu_i(o_i^j) \nabla_{a_i} Q_i^{\mu}(\mathbf{x}^j, a_1^j, \dots, a_i, \dots, a_N^j) |_{a_i = \mu_i(o_i^j)}$$

    end for
    Update target network parameters for each agent  $i$ :
      
$$\theta_i' \leftarrow \tau \theta_i + (1 - \tau) \theta_i'$$

  end for
end for

```

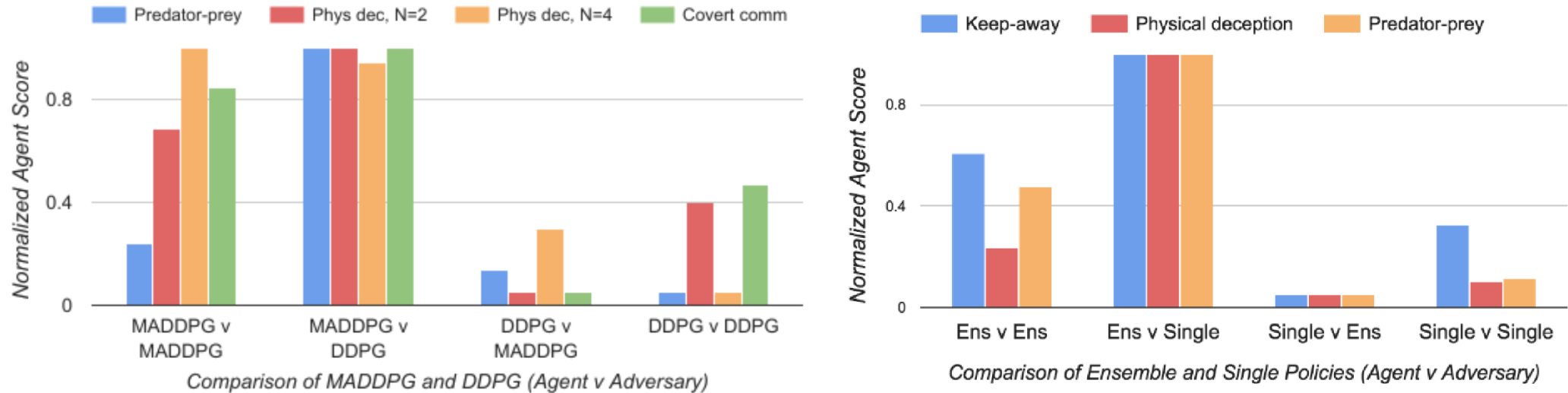
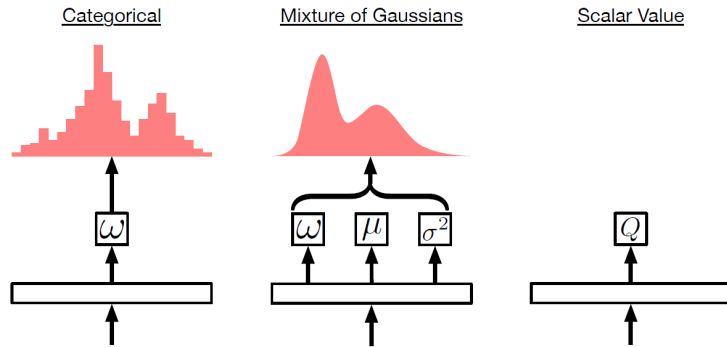


Figure 3: Comparison between MADDPG and DDPG (left), and between single policy MADDPG and ensemble MADDPG (right) on the competitive environments. Each bar cluster shows the 0-1 normalized score for a set of competing policies (agent v adversary), where a higher score is better for the agent. In all cases, MADDPG outperforms DDPG when directly pitted against it, and similarly for the ensemble against the single MADDPG policies. Full results are given in the Appendix.

Distributed Distributional Deep Deterministic Policy Gradient (D4PG)



a distributional version of the critic update.



distributed parallel actors

N-step returns; prioritization of the experience replay

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, p_i = |\delta_i| + \epsilon$$

Algorithm 1 D4PG

Input: batch size M , trajectory length N , number of actors K , replay size R , exploration constant ϵ , initial learning rates α_0 and β_0

- 1: Initialize network weights (θ, w) at random
- 2: Initialize target weights $(\theta', w') \leftarrow (\theta, w)$
- 3: Launch K actors and replicate network weights (θ, w) to each actor
- 4: **for** $t = 1, \dots, T$ **do**
- 5: Sample M transitions $(\mathbf{x}_{i:i+N}, \mathbf{a}_{i:i+N-1}, r_{i:i+N-1})$ of length N from replay with priority p_i
- 6: Construct the target distributions $Y_i = \left(\sum_{n=0}^{N-1} \gamma^n r_{i+n} \right) + \gamma^N Z_{w'}(\mathbf{x}_{i+N}, \pi_{\theta'}(\mathbf{x}_{i+N}))$
Note, although not denoted the target Y_i may be projected (e.g. for Categorical value distributions).
- 7: Compute the actor and critic updates

$$\delta_w = \frac{1}{M} \sum_i \nabla_w (Rp_i)^{-1} d(Y_i, Z_w(\mathbf{x}_i, \mathbf{a}_i))$$

$$\delta_\theta = \frac{1}{M} \sum_i \nabla_\theta \pi_\theta(\mathbf{x}_i) \mathbb{E}[\nabla_{\mathbf{a}} Z_w(\mathbf{x}_i, \mathbf{a})] \big|_{\mathbf{a}=\pi_\theta(\mathbf{x}_i)}$$
- 8: Update network parameters $\theta \leftarrow \theta + \alpha_t \delta_\theta, w \leftarrow w + \beta_t \delta_w$
- 9: If $t = 0 \bmod t_{\text{target}}$, update the target networks $(\theta', w') \leftarrow (\theta, w)$
- 10: If $t = 0 \bmod t_{\text{actors}}$, replicate network weights to the actors
- 11: **end for**
- 12: **return** policy parameters θ

Actor

- 1: **repeat**
- 2: Sample action $\mathbf{a} = \pi_\theta(\mathbf{x}) + \epsilon \mathcal{N}(0, 1)$
- 3: Execute action $\mathbf{a}$, observe reward r and state $\mathbf{x}'$
- 4: Store $(\mathbf{x}, \mathbf{a}, r, \mathbf{x}')$ in replay
- 5: **until** learner finishes

Barth-Maron, Gabriel, Matthew W. Hoffman, David Budden, Will Dabney, Dan Horgan, Dhruva Tb, Alistair Muldal, Nicolas Heess, and Timothy Lillicrap. "Distributed distributional deterministic policy gradients." ICLR 2018.

Bellemare, Marc G., Will Dabney, and Rémi Munos. "A distributional perspective on reinforcement learning." In International conference on machine learning, pp. 449-458. PMLR, 2017.

Schaul, Tom, John Quan, Ioannis Antonoglou, and David Silver. "Prioritized experience replay." ICLR 2016.

<https://zhuanlan.zhihu.com/p/137935717>

Distributed Distributional Deep Deterministic Policy Gradient (D4PG)

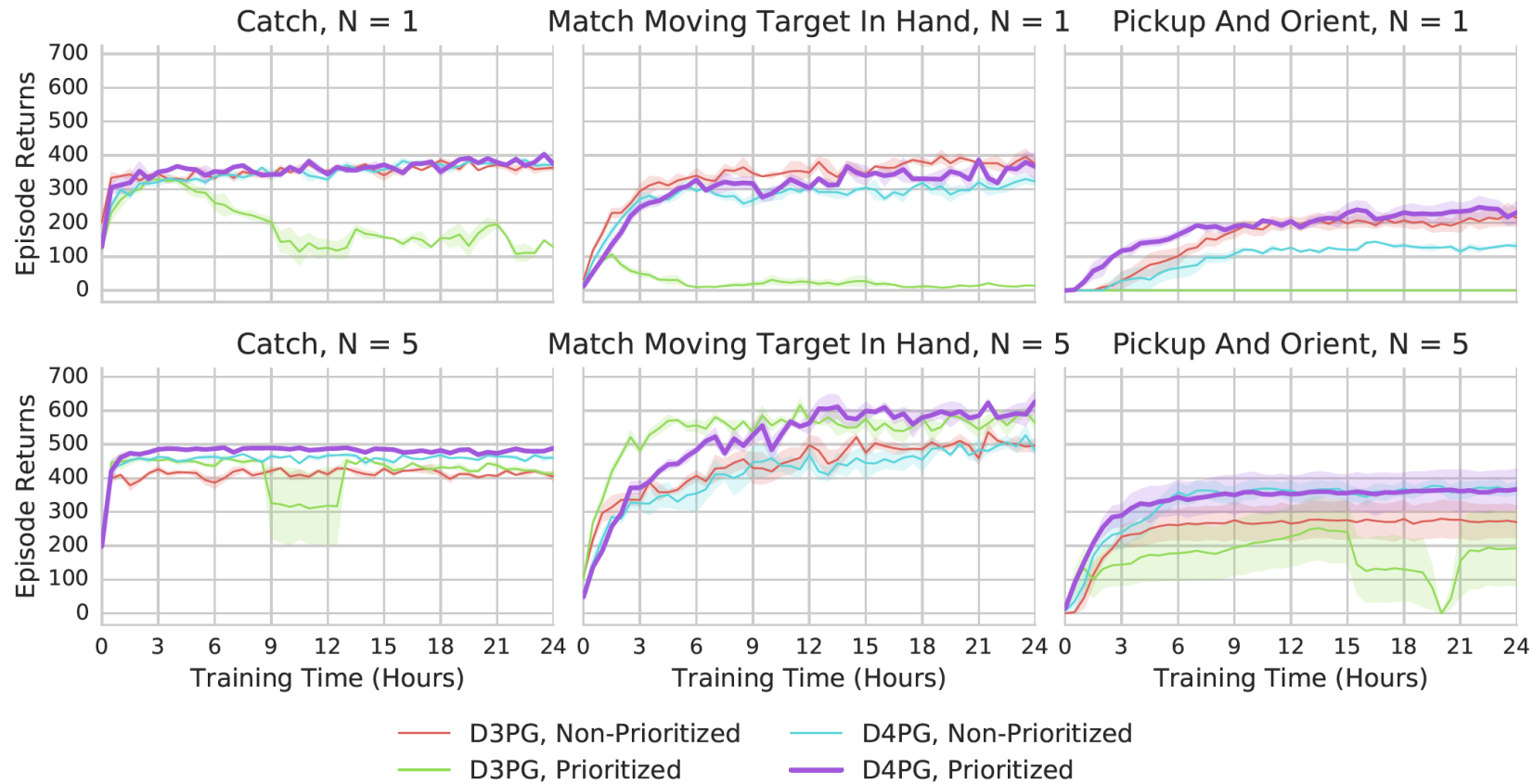


Figure 3: Experimental results for tasks in the manipulation domain.

Truncated Quantile Critics (TQC)

Controlling overestimation bias with truncated mixture of continuous distributional quantile critics.

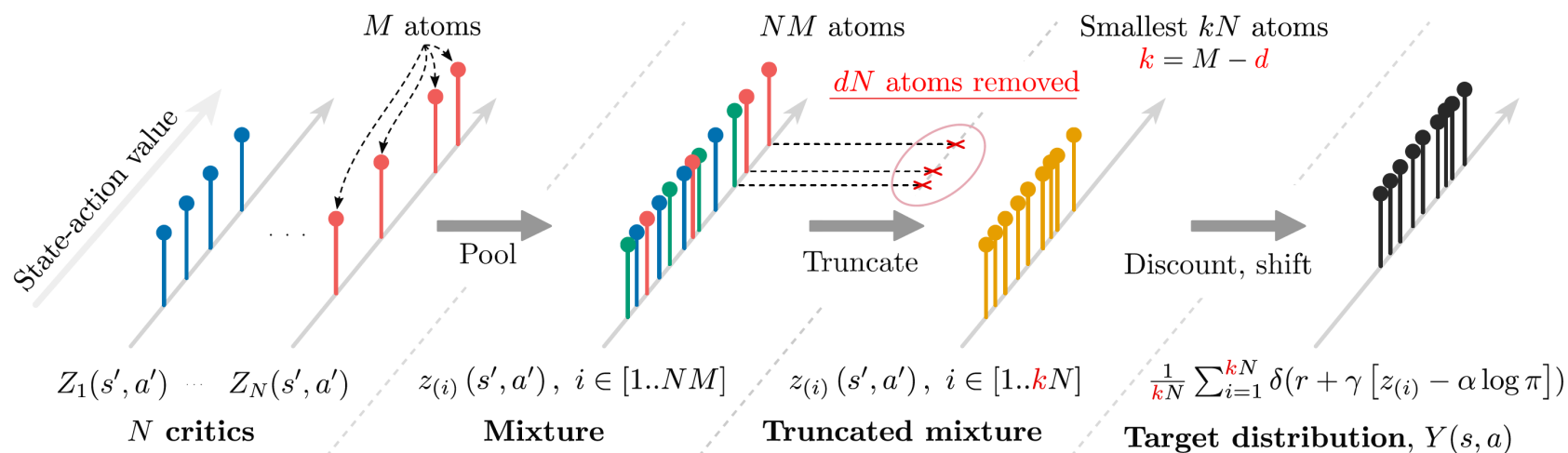


Figure 2. Step-by-step construction of the temporal difference target distribution $Y(s, a)$. First, we compute approximations of the return distribution conditioned on s' and a' by evaluating N separate target critics. Second, we make a mixture out of the N distributions from the previous step. Third, we truncate the right tail of this mixture to obtain atoms $z_{(i)}(s', a')$ from equation 11. Fourthly, we add entropy term, discount and add reward as in soft Bellman equation.

distributional representation
of a critic (M=25 atoms)

ensemble of multiple
critics (N=5)

truncation of mixed critics prediction

- truncate target return distribution at the value learning stage
truncate the right tail of the return distribution approximation by dropping several of the topmost atoms, averaging the locations of the remaining atoms
- use nontruncated critics' predictions for policy optimization

Truncated Quantile Critics (TQC)

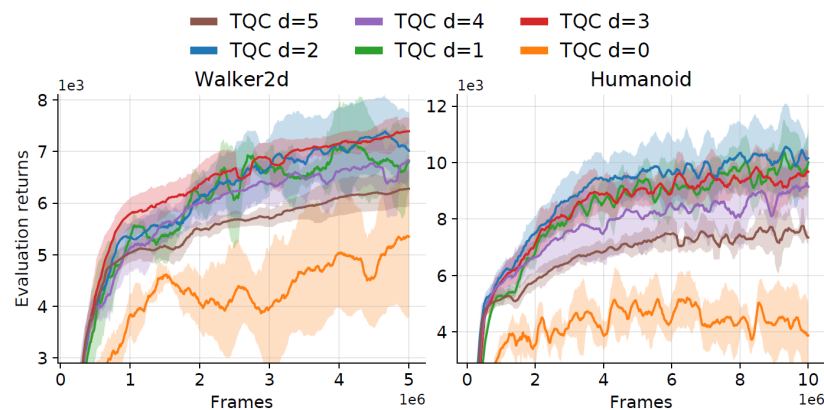


Figure 8. Varying the number of dropped atoms per critic d . $N = 5$ networks, $M = 25$ atoms. Smoothed with a window of 200, $\pm$ std is plotted.

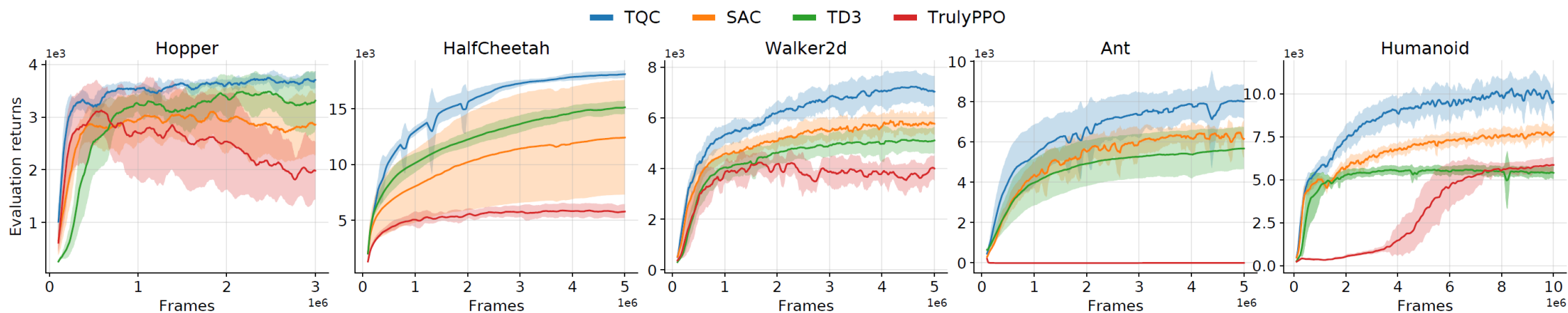


Figure 5. Average performances of methods on MuJoCo Gym Environments with $\pm$ std shaded. Smoothed with a window of 100.

Data-regularized Q (DrQ)

Enabling robust learning directly from pixels without the need for auxiliary losses or pre-training.

random shifts

- transformations of the input image
pad each side by 4 pixels (by repeating boundary pixels) and then select a random 84x84 crop

- averaging the Q target over K image transformations

$$y_i = r_i + \gamma \frac{1}{K} \sum_{k=1}^K Q_{\theta}(f(s'_i, v'_{i,k}), a'_{i,k}), a'_{i,k} \sim \pi(\cdot | f(s'_i, v'_{i,k}))$$

- averaging the Q function itself over M image transformations

$$\theta \leftarrow \theta - \lambda_{\theta} \nabla_{\theta} \frac{1}{NM} \sum_{i=1, m=1}^{N, M} (Q_{\theta}(f(s_i, v_{i,m}), a_i) - y_i)^2$$

Algorithm 1 DrQ: Data-regularized Q applied to a generic off-policy actor critic algorithm.

Black: unmodified off-policy actor-critic.

Orange: image transformation.

Green: target Q augmentation.

Blue: Q augmentation.

Hyperparameters: Total number of environment steps T , mini-batch size N , learning rate λ_{θ} , target network update rate τ , image transformation f , number of target Q augmentations K , number of Q augmentations M .

for each timestep $t = 1..T$ **do**

$a_t \sim \pi(\cdot | s_t)$

$s'_t \sim p(\cdot | s_t, a_t)$

$\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r(s_t, a_t), s'_t)$

UPDATECRITIC($\mathcal{D}$)

UPDATEACTOR($\mathcal{D}$) $\triangleright$ Data augmentation is applied to the samples for actor training as well.

end for

procedure UPDATECRITIC($\mathcal{D}$)

$\{(s_i, a_i, r_i, s'_i)\}_{i=1}^N \sim \mathcal{D}$

$\triangleright$ Sample a mini batch

$\{\nu'_{i,k} | \nu'_{i,k} \sim \mathcal{U}(\mathcal{T}), i = 1..N, k = 1..K\}$

$\triangleright$ Sample parameters of target augmentations

for each $i = 1..N$ **do**

$a'_i \sim \pi(\cdot | s'_i)$ or $a'_{i,k} \sim \pi(\cdot | f(s'_i, \nu'_{i,k})), k = 1..K$

$\hat{Q}_i = Q_{\theta'}(s'_i, a'_i)$ or $\hat{Q}_i = \frac{1}{K} \sum_{k=1}^K Q_{\theta'}(f(s'_i, \nu'_{i,k}), a'_{i,k})$

$y_i \leftarrow r(s_i, a_i) + \gamma \hat{Q}_i$

end for

$\{\nu_{i,m} | \nu_{i,m} \sim \mathcal{U}(\mathcal{T}), i = 1..N, m = 1..M\}$

$\triangleright$ Sample parameters of Q augmentations

$J_Q(\theta) = \frac{1}{N} \sum_{i=1}^N (Q_{\theta}(s_i, a_i) - y_i)^2$ or $J_Q(\theta) = \frac{1}{NM} \sum_{i=1, m=1}^{N, M} (Q_{\theta}(f(s_i, \nu_{i,m}), a_i) - y_i)^2$

$\theta \leftarrow \theta - \lambda_{\theta} \nabla_{\theta} J_Q(\theta)$

$\triangleright$ Update the critic

$\theta' \leftarrow (1 - \tau)\theta' + \tau\theta$

$\triangleright$ Update the critic target

end procedure

Data-regularized Q (DrQ)

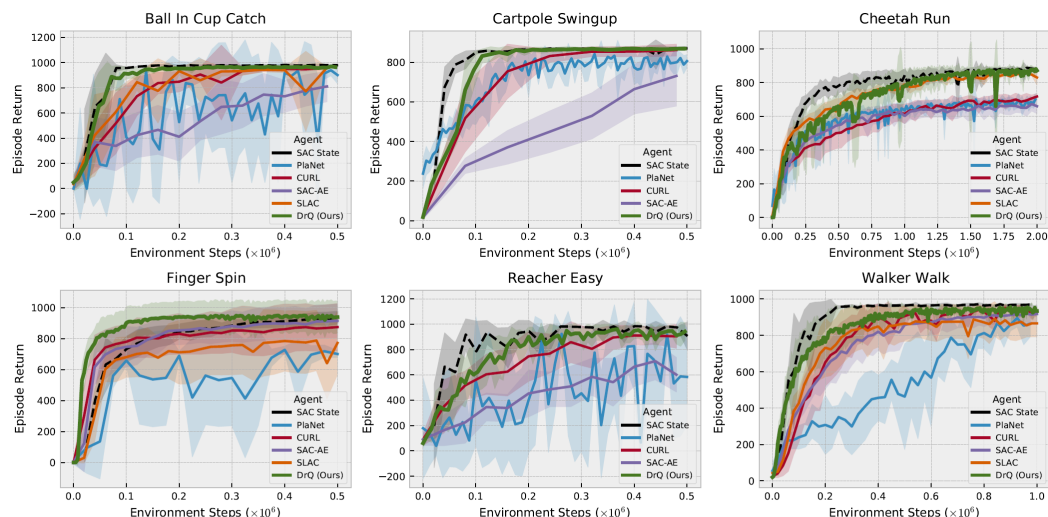


Figure 3: The PlaNet benchmark. Our algorithm (**DrQ** [$K=2, M=2$]) outperforms the other methods and demonstrates the state-of-the-art performance. Furthermore, on several tasks **DrQ** is able to match the upper-bound performance of SAC trained directly on internal state, rather than images. Finally, our algorithm not only shows improved sample-efficiency relative to other approaches, but is also faster in terms of wall clock time.

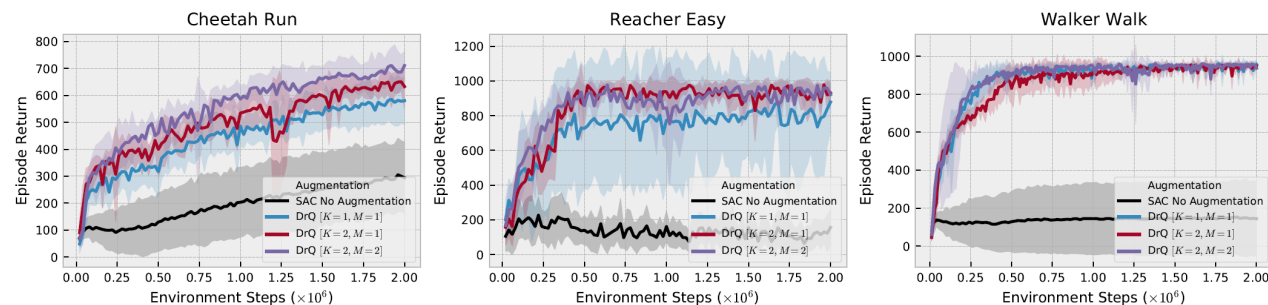
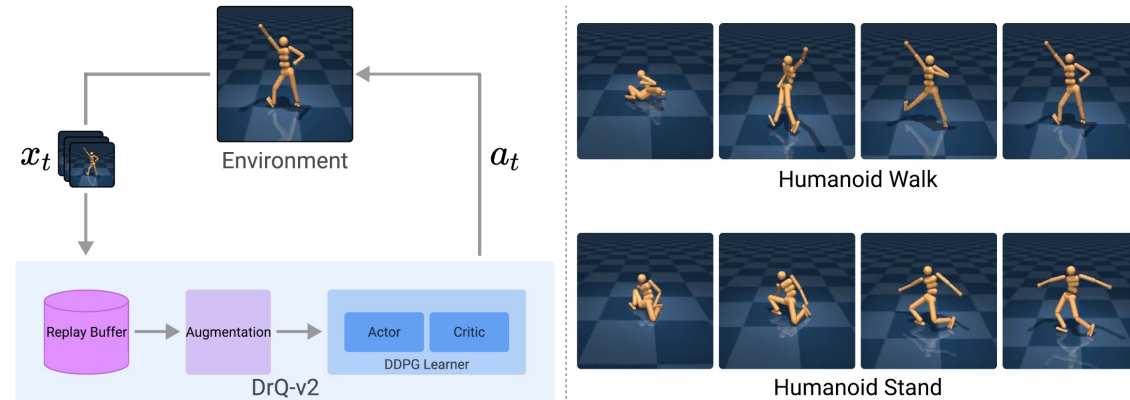


Figure 2: Different combinations of our three regularization techniques on tasks from [51] using SAC. Black: standard SAC. Blue: **DrQ** [$K=1, M=1$], SAC augmented with random shifts. Red: **DrQ** [$K=2, M=1$], random shifts + Target Q augmentations. Purple: **DrQ** [$K=2, M=2$], random shifts + Target Q + Q augmentations. All three regularization methods correspond to Algorithm 1 with different hyperparameters K, M and independently provide beneficial gains over unaugmented SAC. Note that **DrQ** [$K=1, M=1$] exactly recovers the concurrent work of RAD [36] up to a particular choice of hyper-parameters and data augmentation type.

DrQ-v2

Improves upon DrQ.
Able to solve complex humanoid locomotion tasks directly from pixel observations.
Simple and fast.



random shifts

- padding each side of 84x84 observation rendering by 4 pixels, and then selecting a random 84x84 crop
- bilinear interpolation: replace each pixel value with the average of the four nearest pixel values**

DDPG

- switching the base RL algorithm from SAC to DDPG (also some TD3 tricks like clipped double Q-learning)
- allows us straightforwardly incorporating multi-step return

exploration schedule

- linear decay $\sigma(t)$ of the exploration noise:
$$\sigma(t) = \sigma_{\text{final}} + (1 - \min(\frac{t}{T}, 1))(\sigma_{\text{init}} - \sigma_{\text{final}})$$

better parameters

- 10 times larger replay buffer (1e6) than DrQ (1e5)
- smaller mini-batch size of 256 (512 in DrQ)
- smaller learning rate of 1e-4 (1e-3 in DrQ)

Algorithm 1 DrQ-v2: Improved data-augmented RL.

Inputs:

$f_\xi, \pi_\phi, Q_{\theta_1}, Q_{\theta_2}$: parametric networks for encoder, policy, and Q-functions respectively.
 aug: random shifts image augmentation.
 $\sigma(t)$: scheduled standard deviation for the exploration noise defined in Equation (3).
 T, B, α, τ, c : training steps, mini-batch size, learning rate, target update rate, clip value.

Training routine:

for each timestep $t = 1..T$ **do**

$\sigma_t \leftarrow \sigma(t)$
 $\mathbf{a}_t \leftarrow \pi_\phi(f_\xi(\mathbf{x}_t)) + \epsilon$ and $\epsilon \sim \mathcal{N}(0, \sigma_t^2)$
 $\mathbf{x}_{t+1} \sim P(\cdot | \mathbf{x}_t, \mathbf{a}_t)$
 $\mathcal{D} \leftarrow \mathcal{D} \cup (\mathbf{x}_t, \mathbf{a}_t, R(\mathbf{x}_t, \mathbf{a}_t), \mathbf{x}_{t+1})$
 UPDATECRITIC($\mathcal{D}, \sigma_t$)
 UPDATEACTOR($\mathcal{D}, \sigma_t$)

end for

procedure UPDATECRITIC($\mathcal{D}, \sigma$)

$\{(\mathbf{x}_t, \mathbf{a}_t, r_{t:t+n-1}, \mathbf{x}_{t+n})\} \sim \mathcal{D}$
 $\mathbf{h}_t, \mathbf{h}_{t+n} \leftarrow f_\xi(\text{aug}(\mathbf{x}_t)), f_\xi(\text{aug}(\mathbf{x}_{t+n}))$
 $\mathbf{a}_{t+n} \leftarrow \pi_\phi(\mathbf{h}_{t+n}) + \epsilon$ and $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma^2))$
 Compute $\mathcal{L}_{\theta_1, \xi}$ and $\mathcal{L}_{\theta_2, \xi}$ using Equation (1)
 $\xi \leftarrow \xi - \alpha \nabla_\xi (\mathcal{L}_{\theta_1, \xi} + \mathcal{L}_{\theta_2, \xi})$
 $\theta_k \leftarrow \theta_k - \alpha \nabla_{\theta_k} \mathcal{L}_{\theta_k, \xi} \quad \forall k \in \{1, 2\}$
 $\bar{\theta}_k \leftarrow (1 - \tau) \bar{\theta}_k + \tau \theta_k \quad \forall k \in \{1, 2\}$

end procedure

procedure UPDATEACTOR($\mathcal{D}, \sigma$)

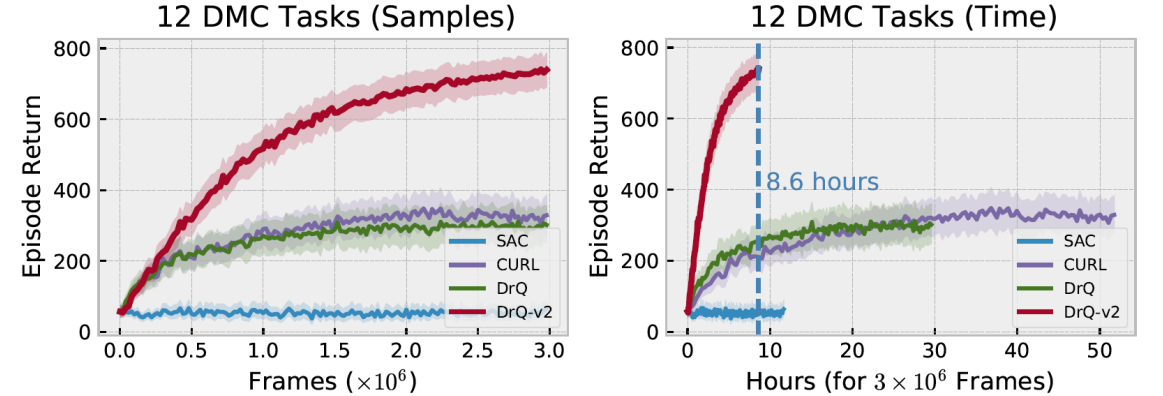
$\{(\mathbf{x}_t)\} \sim \mathcal{D}$
 $\mathbf{h}_t \leftarrow f_\xi(\text{aug}(\mathbf{x}_t))$
 $\mathbf{a}_t \leftarrow \pi_\phi(\mathbf{h}_t) + \epsilon$ and $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma^2))$
 Compute $\mathcal{L}_\phi$ using Equation (2)
 $\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}_\phi$

end procedure

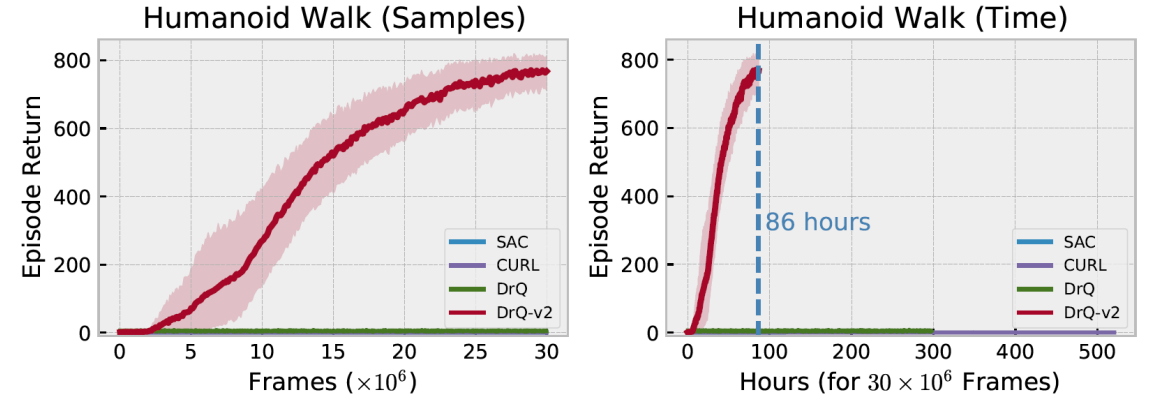
- ▷ Compute stddev for the exploration noise
- ▷ Add noise to the deterministic action
- ▷ Run transition function for one step
- ▷ Add a transition to the replay buffer

- ▷ Sample a mini batch of B transitions
- ▷ Apply data augmentation and encode
 - ▷ Sample action
 - ▷ Compute critic losses
 - ▷ Update encoder weights
 - ▷ Update critic weights
- ▷ Update critic target weights

- ▷ Sample a mini batch of B observations
- ▷ Apply data augmentation and encode
 - ▷ Sample action
 - ▷ Compute actor loss
- ▷ Update actor's weights only



(a) Average performance across 12 challenging DMC tasks.



(b) Performance on *Humanoid Walk*.

Randomized Ensembled Double Q-Learning (REDQ)



high Update-To-Data (UTD) ratio (G=20)

an ensemble of (N=10) Q functions

in-target minimization across a random subset (M=2) of Q

Algorithm 1 Randomized Ensembled Double Q-learning (REDQ)

- 1: Initialize policy parameters θ , **N** Q-function parameters $\phi_i, i = 1, \dots, N$, empty replay buffer $\mathcal{D}$. Set target parameters $\phi_{\text{targ},i} \leftarrow \phi_i$, for $i = 1, 2, \dots, N$
- 2: **repeat**
- 3: Take one action $a_t \sim \pi_\theta(\cdot | s_t)$. Observe reward r_t , new state s_{t+1} .
- 4: Add data to buffer: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$
- 5: **for** **G** updates **do**
- 6: Sample a mini-batch $B = \{(s, a, r, s')\}$ from $\mathcal{D}$
- 7: Sample a set $\mathcal{M}$ of **M** distinct indices from $\{1, 2, \dots, N\}$
- 8: Compute the Q target y (same for all of the N Q-functions):

$$y = r + \gamma \left(\min_{i \in \mathcal{M}} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}' | s') \right), \quad \tilde{a}' \sim \pi_\theta(\cdot | s')$$

- 9: **for** $i = 1, \dots, N$ **do**
- 10: Update ϕ_i with gradient descent using

$$\nabla_{\phi} \frac{1}{|B|} \sum_{(s,a,r,s') \in B} (Q_{\phi_i}(s, a) - y)^2$$

- 11: Update target networks with $\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i$
- 12: Update policy parameters θ with gradient ascent using

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} \left(\frac{1}{N} \sum_{i=1}^N Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s) | s) \right), \quad \tilde{a}_\theta(s) \sim \pi_\theta(\cdot | s)$$

Randomized Ensembled Double Q-Learning (REDQ)

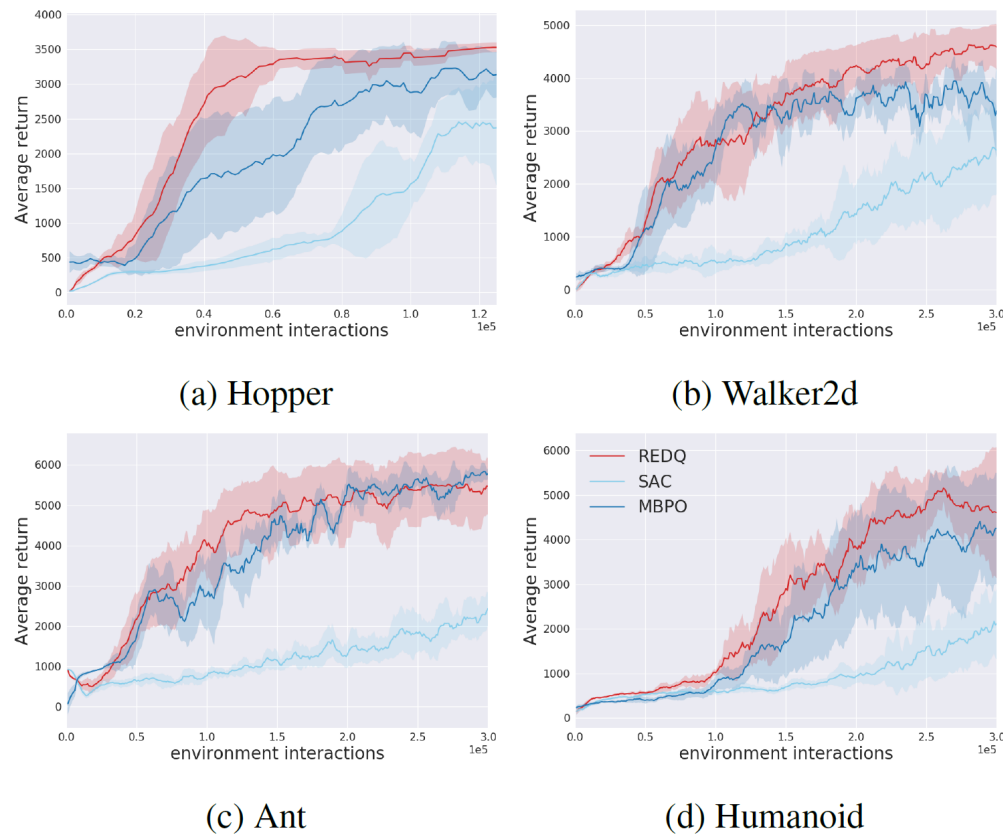


Figure 1: REDQ compared to MBPO and SAC. Both REDQ and MBPO use $G = 20$.

Dropout q-functions (DroQ)

A variant of REDQ that uses a small ensemble of dropout Q-functions.

dropout and ensemble together for Q-functions

layer normalization into dropout Q-functions

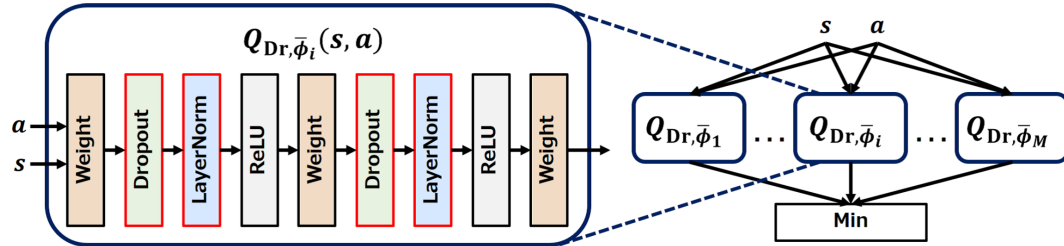


Figure 1: Dropout Q-function implementation (left part) and how dropout Q-functions are used in target (right part). **Dropout Q-function implementation:** Our dropout function is implemented by modifying that used by Chen et al. (2021b). Our modification (highlighted in red) is adding dropout (Dropout) and layer normalization (LayerNorm). “Weight” is a weight layer and “ReLU” is the activation layer of rectified linear units. Parameters $\bar{\phi}_i$ represent the weights and biases in weight layers. **How dropout Q-functions are used in target:** M dropout Q-functions are used to calculate the target value as $\min_{i=1, \dots, M} Q_{\text{Dr}, \bar{\phi}_i}(s, a)$.

Algorithm 2 DroQ

- 1: Initialize policy parameters θ , M Q-function parameters ϕ_i , $i = 1, \dots, M$, and empty replay buffer $\mathcal{D}$. Set target parameters $\bar{\phi}_i \leftarrow \phi_i$, for $i = 1, \dots, M$.
- 2: **repeat**
- 3: Take action $a_t \sim \pi_\theta(\cdot|s_t)$. Observe reward r_t , next state s_{t+1} ; $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r_t, s_{t+1})$.
- 4: **for** G updates **do**
- 5: Sample a mini-batch $\mathcal{B} = \{(s, a, r, s')\}$ from $\mathcal{D}$.
- 6: Compute the Q target y for the dropout Q-functions:

$$y = r + \gamma \left(\min_{i=1, \dots, M} Q_{\text{Dr}, \bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a'|s') \right), \quad a' \sim \pi_\theta(\cdot|s')$$

- 7: **for** $i = 1, \dots, M$ **do**
- 8: Update ϕ_i with gradient descent using

$$\nabla_{\phi} \frac{1}{|\mathcal{B}|} \sum_{(s, a, r, s') \in \mathcal{B}} (Q_{\text{Dr}, \phi_i}(s, a) - y)^2$$

- 9: Update target networks with $\bar{\phi}_i \leftarrow \rho \bar{\phi}_i + (1 - \rho) \phi_i$.
- 10: Update θ with gradient ascent using

$$\nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \left(\frac{1}{M} \sum_{i=1}^M Q_{\text{Dr}, \phi_i}(s, a) - \alpha \log \pi_\theta(a|s) \right), \quad a \sim \pi_\theta(\cdot|s)$$

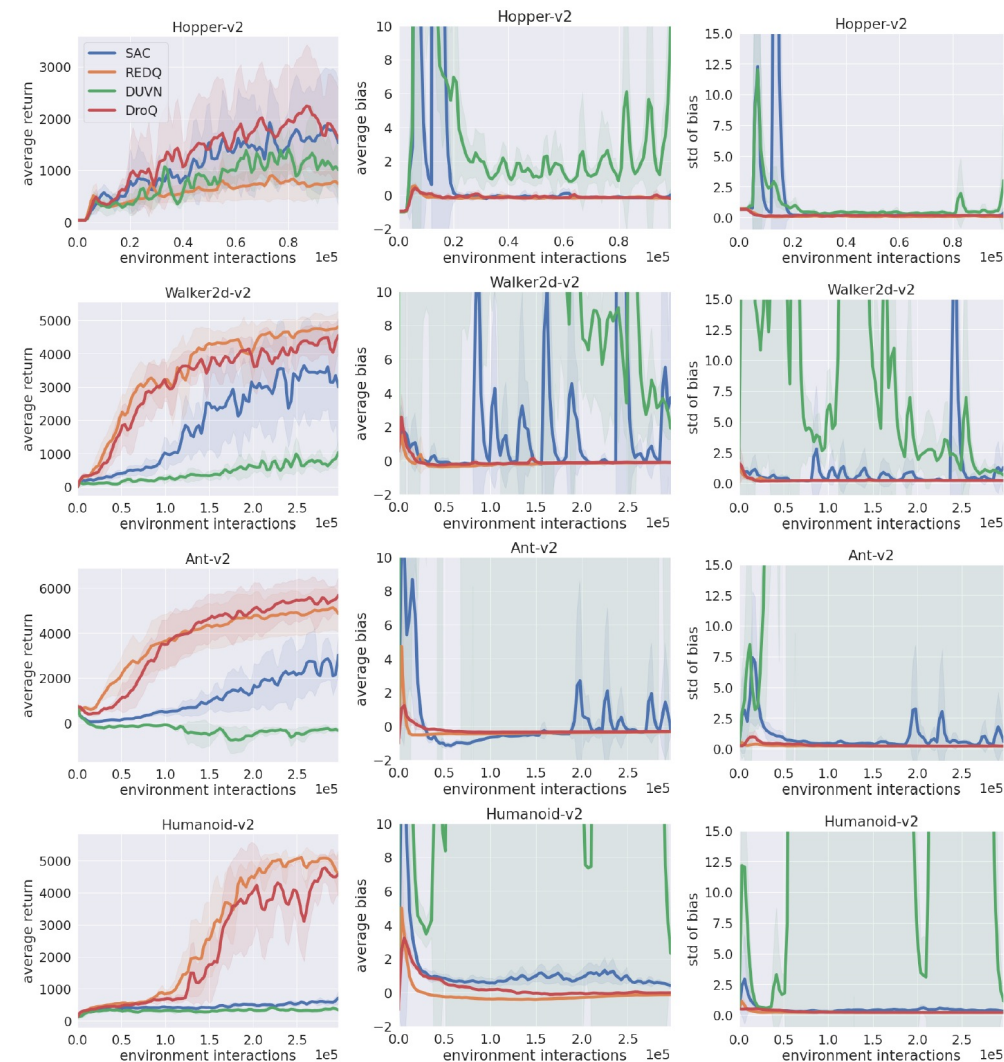
Dropout q-functions (DroQ)

Table 1: Process times (in msec) per overall loop (e.g., lines 3–10 in Algorithm 2). Process times per Q-functions update (e.g., lines 4–9 in Algorithm 2) are shown in parentheses.

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
SAC	910 (870)	870 (835)	888 (848)	893 (854)
REDQ	2328 (2269)	2339 (2283)	2336 (2277)	2400 (2340)
DUVN	664 (636)	762 (731)	733 (700)	692 (660)
DroQ	948 (905)	933 (892)	954 (913)	989 (946)

$G=20$, $N=10$, $M=2$

SAC: delayed policy update and high UTD ratio $G = 20$ were used



state-action learned embeddings (SALE)

learning embeddings that model the nuanced interaction between state and action

- representation learning with low-level states
- difficulty of a task is often defined by the complexity of the underlying dynamical system, rather than the size of the observation space

Policy Checkpoints

fixing each policy for multiple training episodes

at test time, using the highest-performing policy observed during training

prioritized experience replay

behavior cloning term

used only for offline RL

state-action learned embeddings (SALE)

- a pair of encoders (f, g) , dynamics prediction loss:

$$\mathcal{L}(f, g) := (g(f(s), a) - |f(s')|_{\times})^2$$

- normalized embeddings

$$\text{AvgL1Norm}(x) := \frac{x}{\frac{1}{N} \sum_i |x_i|}$$

- concatenate the embeddings with the original state and action
- clipped values to control extrapolation error

$$Q_{t+1}(s, a) \approx r + \gamma \text{clip}(Q_t(s', a'), \min_{(s,a) \in D} Q_t(s, a), \max_{(s,a) \in D} Q_t(s, a))$$

- gradients from the value function and policy are not propagated to (f, g)

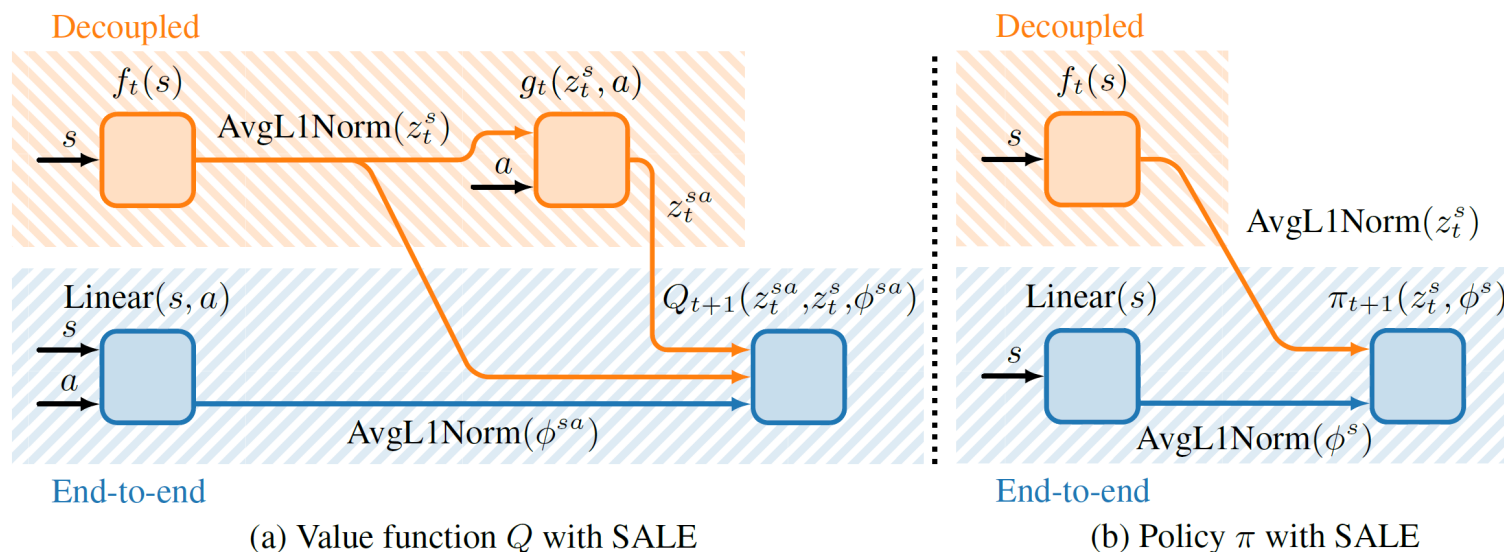


Figure 1: **Diagram of State-Action Learned Embeddings (SALE).** SALE uses encoders (f, g) to output embeddings (z^s, z^{sa}) to enhance the input of the value function Q and policy π . ϕ denotes the output of the corresponding linear layer. ■ The encoders (f, g) are jointly trained to predict the next state embedding (where $|\cdot|_{\times}$ denotes the stop-gradient operation), decoupled from the training of the value function and policy (Equation 2). ■ The end-to-end linear layers are trained with gradients from the corresponding network. AvgL1Norm is used to keep the scale of each of the inputs to the value function and policy constant.

Policy Checkpoints

Standard off-policy RL: Collect a data point -> train once

Proposed: Collect N data points over several assessment episodes -> train N times

At evaluation time, the checkpoint policy is used, rather than the current policy

prioritized experience replay

LAP

$$p(i) = \frac{\max(|\delta(i)|^\alpha, 1)}{\sum_{j \in D} \max(|\delta(j)|^\alpha, 1)}, \delta(i) := Q(s, a) - y$$

PER

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, p_i = |\delta_i| + \epsilon$$

behavior cloning term (offline)

TD3+BC

$$\pi \approx \operatorname{argmax}_{\pi} \mathbb{E}_{(s,a) \sim D} [Q(s, \pi(s)) - \lambda \|\mathbb{E}_{s \sim D} [Q(s, \pi(s))] - \pi(s) - a\|_\times^2]$$

Algorithm 1 Online TD7

```

1: Initialize: ▷ Before training
   · Policy  $\pi_{t+1}$ , value function  $Q_{t+1}$ , encoders  $(f_{t+1}, g_{t+1})$ .
   · Target policy  $\pi_t$ , target value function  $Q_t$ , fixed encoders  $(f_t, g_t)$ , target fixed encoders  $(f_{t-1}, g_{t-1})$ .
   · Checkpoint policy  $\pi_c$ , checkpoint encoder  $f_c$ .

2: for episode = 1 to final_episode do ▷ Data collection
3:   Using current policy  $\pi_{t+1}$ , collect transitions and store in the LAP replay buffer.

4:   if checkpoint_condition then ▷ Checkpointing
5:     if actor  $\pi_{t+1}$  outperforms checkpoint policy  $\pi_c$  then
6:       Update checkpoint networks  $\pi_c \leftarrow \pi_{t+1}$ ,  $f_c \leftarrow f_t$ .

7:   for  $i = 1$  to timesteps_since_training do ▷ Training
8:     Sample transitions from LAP replay buffer (Equation 10).
9:     Train encoder (Equation 2), value function (Equations 6 & 9), and policy (Equation 11).
10:    if target_update_frequency steps have passed then
11:      Update target networks (Equation 8).

▷ Detailed hyperparameter explanations found in the Appendix.

```

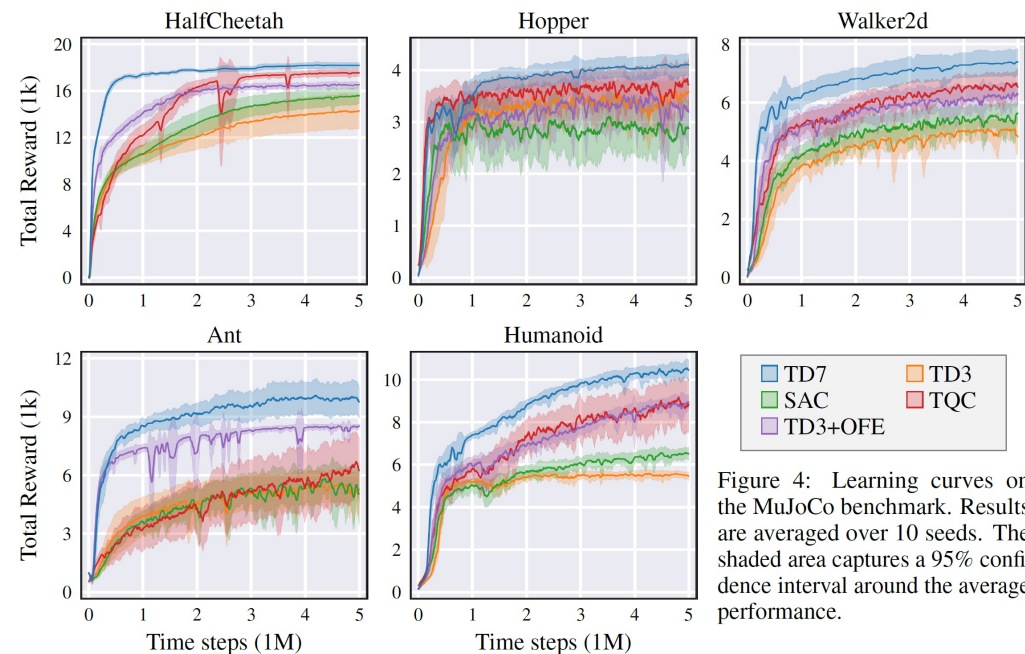
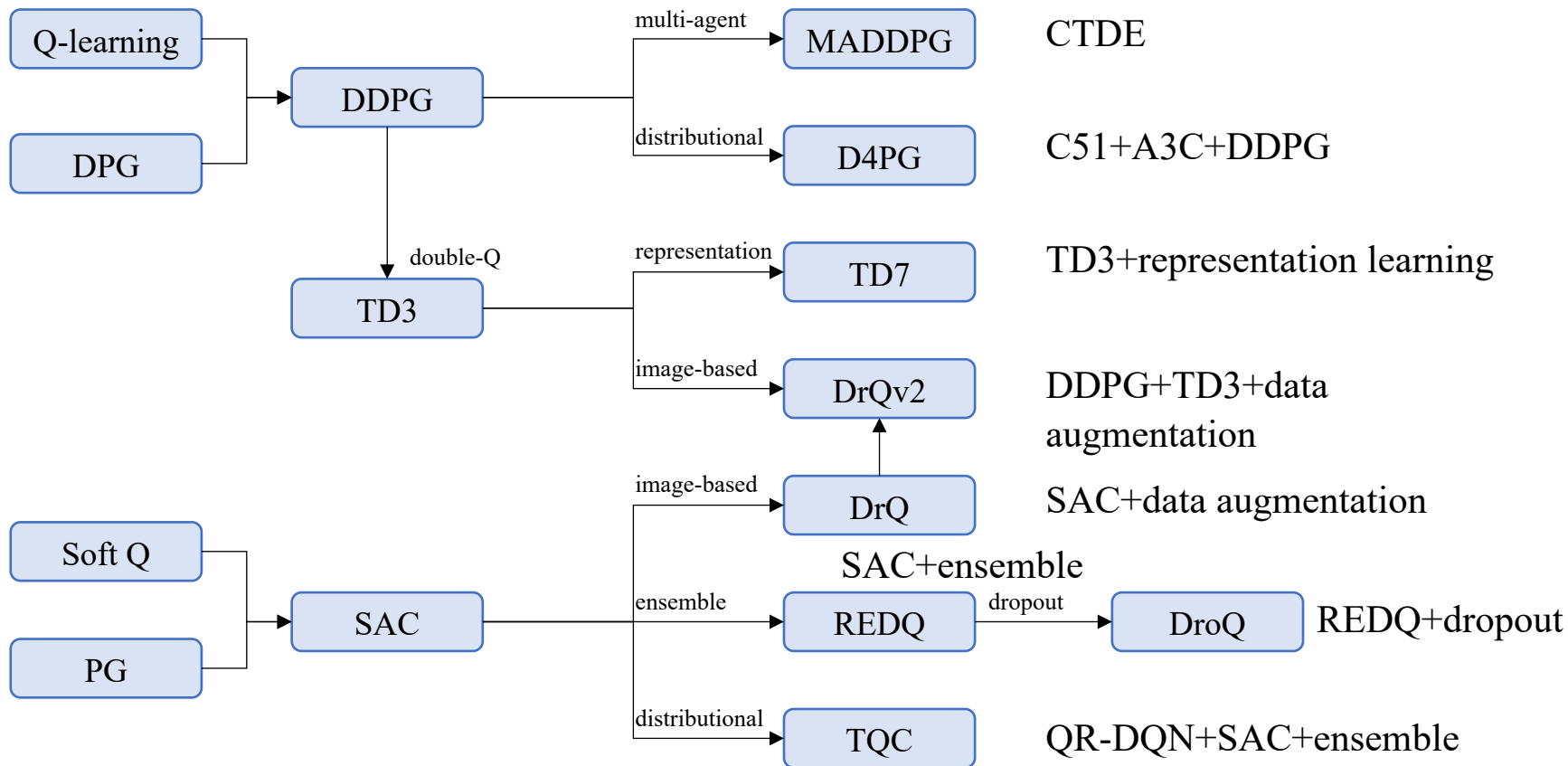


Table 1: Average performance on the MuJoCo benchmark at 300k, 1M, and 5M time steps, over 10 trials, where $\pm$ captures a 95% confidence interval. The highest performance is highlighted. Any performance which is not statistically significantly worse than the highest performance (according to a Welch’s t -test with significance level 0.05) is highlighted.

Discussions



Estimate Q function

Overestimation/extrapolation Error

- Double Q
- Ensemble Q
- Distributional Q

Overfitting/generalization

- Data augmentation
- Representation learning

Others?

- Update-To-Data (UTD) ratio?
- Clip?
- Target network?



THANKS!